arXiv:2603.24517v1 [cs.LG] 25 Mar 2026

# AVO: Agentic Variation Operators for Autonomous Evolutionary Search

**Terry Chen**[*], **Zhifan Ye**[*], **Bing Xu**[*], **Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen**
**Andrew Kerr, Haicheng Wu, Yang Xu, Yu-Jung Chen, Hanfeng Chen, Aditya Kane**
**Ronny Krashinsky, Ming-Yu Liu, Vinod Grover, Luis Ceze, Roger Bringmann,**
**John Tran, Wei Liu, Fung Xie, Michael Lightstone, Humphrey Shi**
NVIDIA

## Abstract

Agentic Variation Operators (AVO) are a new family of evolutionary variation operators that replace the fixed mutation, crossover, and hand-designed heuristics of classical evolutionary search with autonomous coding agents. Rather than confining a language model to **candidate generation within a prescribed pipeline**, AVO instantiates **variation as a self-directed agent loop** that can consult the current lineage, a domain-specific knowledge base, and execution feedback to propose, repair, critique, and verify implementation edits. We evaluate AVO on **attention**, among the most aggressively optimized kernel targets in AI, on NVIDIA Blackwell (B200) GPUs. Over 7 days of continuous autonomous evolution on multi-head attention, AVO discovers kernels that outperform **cuDNN by up to 3.5%** and **FlashAttention-4 by up to 10.5%** across the evaluated configurations. The discovered optimizations transfer readily to grouped-query attention, requiring only **30 minutes** of additional autonomous adaptation and yielding gains of up to **7.0% over cuDNN** and **9.3% over FlashAttention-4**. Together, these results show that agentic variation operators move beyond prior LLM-in-the-loop evolutionary pipelines by elevating the agent from candidate generator to variation operator, and can discover performance-critical micro-architectural optimizations that produce kernels surpassing state-of-the-art expert-engineered attention implementations on today's most advanced GPU hardware.

## 1 Introduction

Large language models have emerged as powerful components in evolutionary search, replacing hand-crafted mutation operators [1] with learned code generation [2–5]. In these systems, an LLM generates candidate solutions conditioned on selected parents, while a surrounding framework, which is usually heuristic-based, handles parent sampling, evaluation, and population management. This combination has produced notable results in mathematical optimization and algorithm discovery, including flagship systems such as FunSearch and AlphaEvolve [3, 4]. However, confining the LLM to candidate generation within a prescribed pipeline fundamentally limits what the LLM can discover: it produces a single output per invocation, with no ability to proactively consult reference materials, test its changes, interpret feedback, or revise its approach before committing a candidate. For the most aggressively hand-tuned implementations, where further improvement requires deep, iterative engineering, this constraint is especially limiting.

We study this problem in the context of attention [6], the central operation in Transformer architectures, and one of the most heavily optimized GPU kernels. The FlashAttention lineage [7–10] and NVIDIA's cuDNN library [11] have pushed attention throughput progressively closer to hardware limits across

[*]Equal Contribution

Preprint.

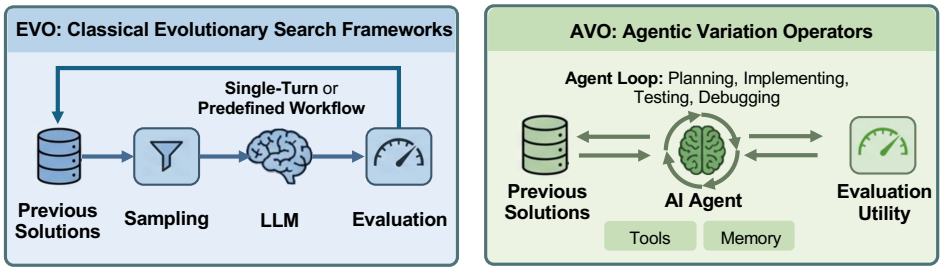


Figure 1: **EVO vs AVO**: Comparison between prior evolutionary search frameworks (e.g. FunSearch, AlphaEvolve, and related LLM-augmented evolutionary approaches) and the proposed Agentic Variation Operator. **Left:** Prior approaches follow a fixed pipeline where the LLM is confined to a single-turn generation step or a predefined workflow, with sampling and evaluation controlled by the framework. **Right:** AVO replaces this pipeline with an autonomous AI agent that iteratively plans, implements, tests, and debugs across long-running sessions, with direct access to previous solutions, evaluation utilities, tools, and persistent memory.

successive GPU generations, with both FlashAttention-4 (FA4) and cuDNN requiring months of manual optimization on the latest Blackwell architecture. Surpassing these implementations demands sustained, iterative interaction with the development environment: studying hardware documentation, analyzing profiler output to identify bottlenecks, implementing and testing candidate optimizations, diagnosing correctness failures, and revising strategy based on accumulated experience.

Recent progress in *deep agents* [12–16] demonstrates that LLMs augmented with planning, persistent memory, and tool use can autonomously navigate such multi-step engineering workflows, with applications ranging from resolving complex GitHub issues to generating key deep learning software [17]. This motivates a fundamentally different role for LLMs in evolutionary search: rather than confining them within a fixed pipeline, we can elevate a deep agent to serve as the variation operator itself. To this end, we propose **Agentic Variation Operators (AVO)**, in which a self-directed coding agent replaces the mutation and crossover process in previous works based on single-turn LLMs [3–5] or fixed workflows [18]. The AVO agent has access to all prior solutions, a domain-specific knowledge base, and the evaluation utility. It autonomously decides what to consult, what to edit, and when to evaluate, enabling continuous improvements over extended time horizons.

To demonstrate its effectiveness, we apply AVO to multi-head attention (MHA) kernels on the Blackwell B200 GPU, and directly compare against the expert-optimized cuDNN and FlashAttention-4 kernels. Over 7 days of continuous evolution without human intervention, the agent explored over 500 optimization directions and evolved 40 kernel versions, producing MHA kernels achieving up to 1668 TFLOPS at BF16 precision, outperforming cuDNN by up to 3.5% and FlashAttention-4 by up to 10.5%. Our analysis of agent-discovered optimizations reveals that they span multiple levels of kernel design, including register allocation, instruction pipeline scheduling, and workload distribution, reflecting genuine hardware-level reasoning. Empirically, we find that the optimization techniques discovered on MHA transfer effectively to grouped-query attention (GQA): adapting the evolved MHA kernel to support GQA requires only 30 minutes of additional autonomous agent effort, yielding up to 7.0% performance improvement over cuDNN and 9.3% over FlashAttention-4.

Our contributions are as follows:

- We introduce Agentic Variation Operators (AVO), a new family of evolutionary variation operators that elevate the agent from candidate generator to variation operator, autonomously exploring domain knowledge, implementing edits, and validating results through iterative interaction with the environment.

- We achieve state-of-the-art MHA throughput on NVIDIA B200 GPUs across the benchmarked configurations, reaching up to 1668 TFLOPS and outperforming cuDNN by up to 3.5% and FlashAttention-4 by up to 10.5%. Furthermore, we show that the discovered optimizations readily transfer to GQA, requiring only 30 minutes of autonomous adaptation and yielding gains of up to 7.0% over cuDNN and 9.3% over FlashAttention-4.

- We provide a detailed analysis of the micro-architectural optimizations discovered by the agent under the benchmarked settings, showing the agent performs genuine hardware-level reasoning rather than superficial code transformations.

# 2 Background

## 2.1 Evolutionary Search and Variation Operators

Evolutionary search optimizes over a space of candidates by maintaining a population $\mathcal{P}$ and iteratively expanding it with new solutions [19]. A population is a set of solution-score pairs $\mathcal{P} = \{(x_i, \mathbf{f}(x_i))\}$, where $\mathbf{f}$ is a scoring function that evaluates each candidate solution. Each iteration produces a new candidate $x_{t+1}$ and updates the population:

$$\mathcal{P}_{t+1} = \texttt{Update}\big(\mathcal{P}_t,\ (x_{t+1},\, \mathbf{f}(x_{t+1}))\big), \quad x_{t+1} = \texttt{Vary}(\mathcal{P}_t), \tag{1}$$

where `Update` adds the new solution to the population, possibly pruning low-score members to maintain a bounded archive. We call `Vary` the *variation operator*: the mechanism by which new candidates are produced from existing ones. In works such as FunSearch [3], AlphaEvolve [4], and related LLM-augmented evolutionary methods [2, 20, 5], the variation operator decomposes into two stages:

$$\texttt{Vary}(\mathcal{P}_t) = \texttt{Generate}\big(\texttt{Sample}(\mathcal{P}_t)\big), \tag{2}$$

where `Sample` selects one or more parent solutions from $\mathcal{P}_t$ (typically guided by score-based and diversity-based heuristics), and `Generate` produces a new candidate conditioned on the sampled parents.

**LLM-augmented variation.** In these approaches, `Generate` is implemented by an LLM that is prompted with the sampled parents and asked to produce a more optimized solution. The `Sample` step, however, remains a fixed algorithmic procedure: AlphaEvolve maintains an island-based evolutionary database inspired by MAP-Elites [21], where a prompt sampler selects parent and inspiration programs using predefined fitness-based and diversity-based heuristics. LoongFlow [18] similarly relies on a MAP-Elites archive with Boltzmann selection for `Sample`, while structuring `Generate` as a fixed Plan-Execute-Summarize pipeline where the LLM sequentially generates a modification plan, produces the code, and summarizes insights. In all these approaches, the LLM only participates in `Generate`: the sampling strategy, evaluation protocol, population management, and the order of operations are all determined by the framework, not by the LLM.

**Learned variation.** TTT-Discover [22] goes further by updating the LLM policy itself through test-time gradient updates, enabling the model to learn an improved `Generate` during the search. Nevertheless, `Sample` remains a fixed algorithm: a PUCT-based selection rule [23] determines which states to expand, and a buffer manages the population with predetermined update rules. Even with a learned `Generate`, the LLM's role is still confined to candidate generation within a rigid algorithmic structure that prescribes when and how it is invoked.

In contrast, the agentic variation operator we introduce in Section 3 replaces the entire `Vary` with a self-directed agent that subsumes `Sample`, `Generate`, and evaluation into a single autonomous loop. The agent has full agency over when to consult reference materials and past solutions $\mathcal{P}_t$, what diagnostic tests to run, and how to revise its optimization strategy.

AVO is orthogonal to the choice of population structure: the agentic operator can in principle be used within archive-based, island-based, or single-lineage evolutionary regimes. In this paper we study the single-lineage setting to isolate the effect of the operator itself.

## 2.2 Attention Kernels on Modern GPUs

**Attention computation.** Given query, key, and value matrices $Q$, $K$, $V$, attention computes $O = \mathrm{softmax}(QK^\top/\sqrt{d})\,V$, where $d$ is the head dimension. A naive implementation materializes the full $N \times N$ score matrix $S = QK^\top$, making the operation memory-bound for large sequence lengths $N$. The FlashAttention algorithm [7] avoids this by computing attention in tiles: it processes key blocks sequentially, maintaining a running softmax (with running row-maximum and row-sum) and accumulating the output $O$ incrementally. This tiling eliminates the need to store the full score matrix, shifting the bottleneck from memory bandwidth to compute throughput on modern GPUs.

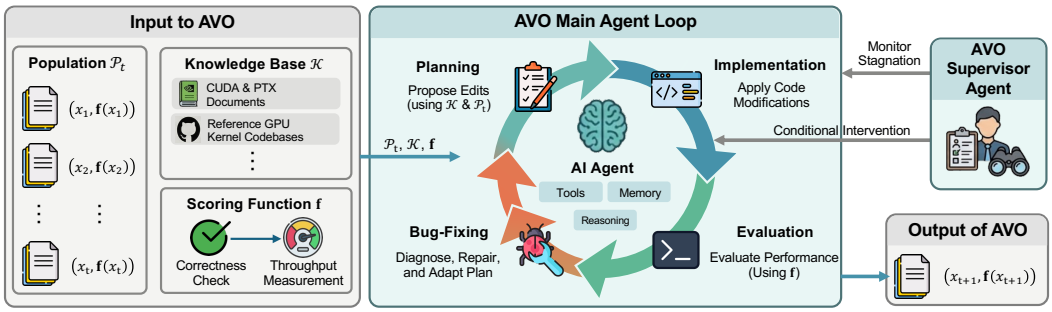


Figure 2: Illustration of the Agentic Variation Operator (AVO).

**Attention kernel on Blackwell hardware.** On NVIDIA's Blackwell architecture, state-of-the-art attention kernels such as FA4 [10] employ *warp specialization*: different warp groups within a thread block are assigned distinct roles in the attention pipeline. *MMA warps* execute the two core matrix multiplications via Blackwell's tensor core instructions: the QK GEMM (producing scores $S$) and the PV GEMM (multiplying the softmax output $P = \mathrm{softmax}(S)$ by $V$ to accumulate the output $O$). *Softmax warps* compute attention weights $P$ from the scores $S$, applying the online softmax algorithm with a running row-maximum. *Correction warps* rescale the output accumulator $O$ when the running maximum changes across K-block iterations (a requirement of the online softmax algorithm). *Load and epilogue warps* handle data movement via the Tensor Memory Accelerator (TMA). In FA4's pipeline, these groups operate concurrently across two Q-tiles (a *dual Q-stage* design), with barrier-based signaling to coordinate handoffs. For causal attention, some K-block iterations are fully masked (no valid attention entries) and others are fully unmasked, leading to different execution paths within the same kernel. With FA4 already representing a highly optimized design, further improvements demand deep hardware expertise, broad exploration across diverse optimization strategies, and repetitive debugging and profiling.

## 3 Agentic Variation Operators

AVO consolidates the sampling, generation, and evaluation stages of evolutionary search into a single autonomous agent run, eliminating the rigid pipeline that constrains existing approaches. Below we formalize this operator, detail what occurs within a single variation step, and describe the mechanism that enables multi-day autonomous exploration.

### 3.1 Formulation

Previous evolutionary search approaches [3, 4] decompose the variation operator as:

$$\texttt{Vary}(\mathcal{P}_t) = \texttt{Generate}(\texttt{Sample}(\mathcal{P}_t)), \tag{3}$$

confining the LLM to the `Generate` step within a fixed pipeline. As illustrated in Figure 2, AVO replaces this decomposition with a single autonomous agent run:

$$\texttt{Vary}(\mathcal{P}_t) = \texttt{Agent}(\mathcal{P}_t, \mathcal{K}, \mathbf{f}), \tag{4}$$

where $\mathcal{P}_t = \{(x_1, \mathbf{f}(x_1)), \ldots, (x_t, \mathbf{f}(x_t))\}$ is the full lineage of solutions and their scores, $\mathcal{K}$ is a domain-specific knowledge base, and $\mathbf{f}$ is the scoring function.

In our setting, each $x_i$ is a CUDA kernel implementation (source code with inline PTX), and $\mathbf{f}$ evaluates a candidate along two dimensions: numerical correctness against a reference implementation, and throughput in TFLOPS on the target hardware. In practice, $\mathbf{f}(x_i) = (f_1(x_i), f_2(x_i), \ldots, f_n(x_i))$ is an $n$-dimensional vector and $f_j$ represents the score for test configuration $j$. A candidate $x_i$ that fails correctness is assigned zero score (i.e., $f_j(x_i) = 0$) regardless of throughput. The knowledge base $\mathcal{K}$ contains CUDA programming guides, PTX ISA documentation, Blackwell architecture specifications, and existing kernel implementations including FlashAttention-4 source code.

AVO defines a family of agentic variation operators for evolutionary search. In this work, we instantiate AVO in a single-lineage autonomous run starting from a seed program $x_0$, producing a

sequence of committed improvements $x_1, x_2, \ldots, x_t$. The accumulated lineage $\mathcal{P}_t$ serves as context for subsequent variation steps.

### 3.2 Anatomy of a Variation Step

A single variation step in AVO, producing $x_{t+1}$ from the current lineage $\mathcal{P}_t$, is an autonomous agent loop. The agent is a general-purpose coding agent with planning, tool use, and persistent memory (details in Section 4), and a single step may involve numerous internal actions.

We observe that the agent frequently examines multiple prior implementations in $\mathcal{P}_t$ within a single variation step, comparing their profiling characteristics to identify bottlenecks and opportunities, and consulting documentation in $\mathcal{K}$ to understand the relevant hardware constraints before implementing a candidate optimization. The agent then invokes $\mathbf{f}$ to test the result. When a candidate fails correctness checks or fails to improve on the current benchmark suite, the agent diagnoses the issue and revises its approach, repeating this edit-evaluate-diagnose cycle until it commits a satisfactory $x_{t+1}$. This design allows the agent to adapt its optimization strategy as the search progresses: early steps may focus on structural changes informed by reference implementations in $\mathcal{K}$, while later steps can shift toward micro-architectural tuning guided by profiling feedback from $\mathbf{f}$ and patterns observed across the accumulated lineage $\mathcal{P}_t$.

In our current implementation, we persist a new committed version only when it passes correctness checks and matches or improves the benchmark score relative to the best committed version so far; unsuccessful intermediate attempts remain part of the agent's internal search trajectory but are not added to the committed lineage.

### 3.3 Continuous Evolution

Although AVO is defined at the level of variation operators for evolutionary search, the present study evaluates a single-lineage continuous instantiation, leaving population-level branching and archive management to future extensions. The AVO agent operates as a continuous loop that periodically produces new solutions without human intervention. Each committed version $x_i$ is persisted as a git commit along with its score, maintaining full state continuity across the entire evolutionary process.

In long-running autonomous optimization, two failure modes can impede progress: the agent may *stall* when it exhausts its current line of exploration, or it may enter *unproductive cycles* of edits that repeatedly fail to improve scores. To mitigate both, AVO incorporates a self-supervision mechanism that detects these scenarios and intervenes. Once triggered, the mechanism reviews the overall evolutionary trajectory and steers the search toward several candidate optimization directions. This conditional intervention effectively redirects exploration with fresh perspective when the current strategy has plateaued.

The 7-day run that produced our final multi-head attention kernel spanned 40 successive versions. Throughout this process, the main agent autonomously decided when to attempt new optimizations, when to revisit earlier approaches in $\mathcal{P}_t$, and when to shift strategy, while the supervisor maintained forward progress by intervening during periods of stagnation.

## 4 Experiments

### 4.1 Setup

**Agent.** We use an internally-developed general-purpose coding agent powered by frontier LLMs as the AVO variation operator. The agent has access to standard software engineering tools, including autonomous code editing, shell command execution, file system navigation, and documentation retrieval. It maintains persistent memory through its conversation history, which accumulates the full context of prior edits, compiler outputs, profiling results, and reasoning across the evolutionary process. No task-specific modifications are made to the agent for kernel optimization; the same agent used for general software engineering tasks is deployed here, with the domain-specific knowledge base $\mathcal{K}$ and scoring function $\mathbf{f}$ provided to the agent as described in Section 3.1.

**Hardware and software.** Following the setup of FA4 [10], all of our experiments are conducted on NVIDIA B200 GPUs with CUDA 13.1 and PyTorch 2.10.0.

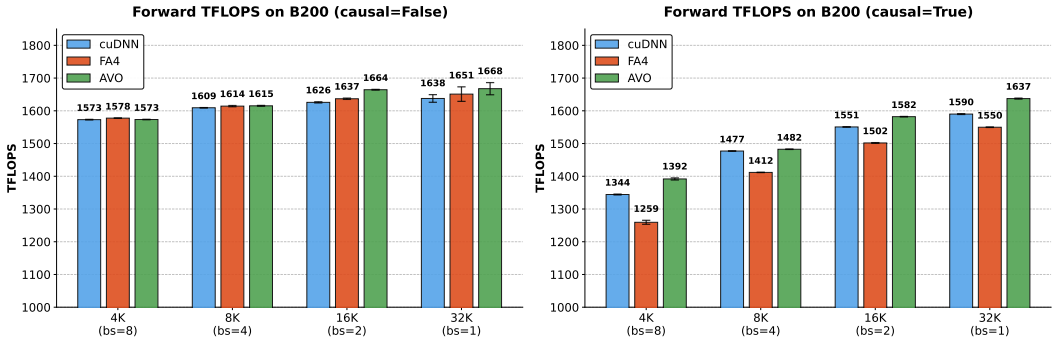


Figure 3: Multi-head attention forward-pass prefilling throughput (TFLOPS) on NVIDIA B200 with head dimension 128, 16 heads, and BF16 precision. Batch size and sequence length are varied with a fixed total of 32k tokens.

**Baselines.** We compare against two state-of-the-art baselines: (1) **cuDNN**: NVIDIA's closed-source attention kernel, measured using cuDNN version 9.19.1, which includes custom optimizations for Blackwell; and (2) **FlashAttention-4** (FA4) [10]: the latest open-source attention kernel optimized for Blackwell, measured using the official implementation (commit `71bf77c`).

**Benchmark Configurations.** We evaluate the forward prefilling throughput with head dimension 128 and BF16 precision across sequence lengths $\{4096, 8192, 16384, 32768\}$. Following FlashAttention-4 [10], we control the total number of tokens to 32768 by adjusting the batch size for each sequence length (e.g., batch size 8 at sequence length 4096, batch size 1 at sequence length 32768). For multi-head attention (MHA), we use 16 heads under both causal and non-causal masking. For grouped-query attention (GQA), we evaluate two configurations drawn from the Qwen3 model family [24]: 32 query heads with 4 KV heads (group size 8, as in Qwen3-30B-A3B) and 32 query heads with 8 KV heads (group size 4, as in Qwen3-8B). For throughput measurement, we used the same timing script from the FA4 repository[2] and the same number of warm-up and repeat rounds as the FA4 paper. In addition, we ran the experiment 10 times to obtain the average performance and the standard deviation. The same setup is used both for agent evolution and for benchmarking the final evolved kernels against the baselines.

### 4.2 Multi-Head Attention

Figure 3 presents the benchmarking results for MHA. On causal attention, AVO outperforms both baselines across all tested configurations, with gains ranging from $+0.4\%$ to $+3.5\%$ over cuDNN and $+5.0\%$ to $+10.5\%$ over FA4. On non-causal attention, AVO achieves modest gains at longer sequences ($+1.8\%$ to $+2.4\%$ over cuDNN at sequence lengths larger than 16384) but is within measurement noise of both baselines at shorter sequences. In Section 4.4, we show how the agent obtains the performance gains through continuous evolution.

### 4.3 Grouped-Query Attention

To evaluate whether agent-discovered optimizations transfer beyond the benchmark settings used in evolution, we prompted the AVO agent to adapt the evolved MHA kernel to support GQA. The agent completed this adaptation autonomously in approximately 30 minutes, producing a GQA-capable kernel without any human guidance on the required changes.

Figure 4 presents the results across two GQA configurations. AVO outperforms both baselines across all configurations. On causal GQA, AVO achieves up to $+7.0\%$ over cuDNN and $+9.3\%$ over FA4. On non-causal GQA, gains reach up to $+6.0\%$ over cuDNN and $+4.5\%$ over FA4. The strong GQA performance demonstrates that the optimizations discovered by the agent during MHA evolution are not specific to the MHA configurations used during evolution, but generalize to the distinct compute and memory access patterns of GQA.

[2] `https://github.com/Dao-AILab/flash-attention/blob/main/benchmarks/benchmark_attn.py`

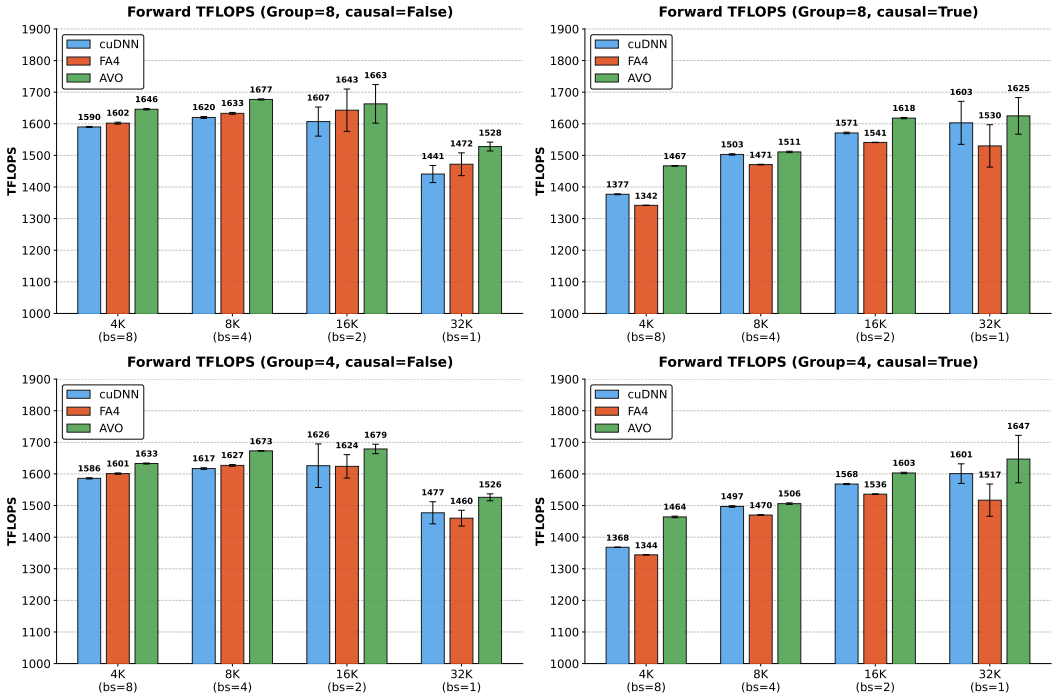


Figure 4: Grouped-query attention forward-pass prefilling throughput (TFLOPS) on NVIDIA B200 with 32 query heads, head dimension 128 and BF16 precision. Results are shown for two GQA configurations (group sizes 8 and 4) under both causal and non-causal masking. The GQA kernel was produced by prompting the AVO agent to adapt the evolved MHA kernel, requiring approximately 30 minutes of autonomous effort.

### 4.4 Evolution Trajectory

In Figure 5 and Figure 6, we show the evolution trajectory of AVO across the 40 committed kernel versions produced during the 7-day evolution. Note that these trajectories visualize the committed sequence, rather than the full internal search tree explored between the commits. We observed the following patterns:

**Scale of exploration.** The 40 committed versions shown in the trajectory represent only the successful outcomes of a much larger search. Over the 7-day evolution, the agent explored over 500 candidate optimization directions internally, including attempts that failed correctness checks, regressed throughput, or were abandoned after profiling. This volume of systematic exploration, each direction requiring reading documentation, implementing changes, compiling, testing, and profiling, far exceeds what a human engineer could accomplish in the same timeframe.

**Discrete jumps rather than gradual improvement.** Throughput improves in distinct steps separated by plateaus where successive versions refine implementation details without measurably changing performance. The five largest gains correspond to architectural inflection points: the introduction of QK-PV interleaving with bitmask causal masking (version 8), a restructured single-pass softmax computation (version 13), the branchless accumulator rescaling with a lighter memory fence for unmasked iterations (version 20), the correction/MMA pipeline overlap (version 30), and register rebalancing across warp groups (version 33). We discuss some of the representative optimizations in Section 5. The remaining versions contribute individually smaller but collectively substantial micro-architectural refinements.

**Diminishing returns.** The earlier versions (v1 through v20) deliver the largest absolute gains per version, closing the gap between a naive implementation and the well-optimized baselines. The later versions (v21 through v40) yield smaller but compounding improvements through cycle-level scheduling and refined resource allocation. This pattern is consistent with the general observation

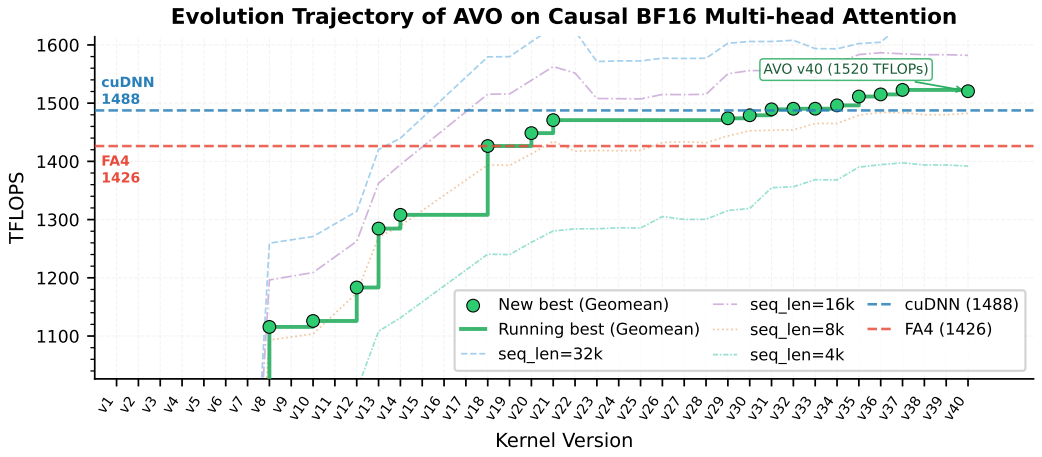


Figure 5: Evolution trajectory of AVO across 40 kernel versions over 7 days on **causal** MHA. The solid green line tracks the running-best geometric mean throughput across all configurations; green circles mark versions that set a new best. Dashed colored lines show per-configuration throughput (seq_len = 4k, 8k, 16k, 32k). Horizontal dashed lines indicate the geometric mean throughput of cuDNN and FA4.

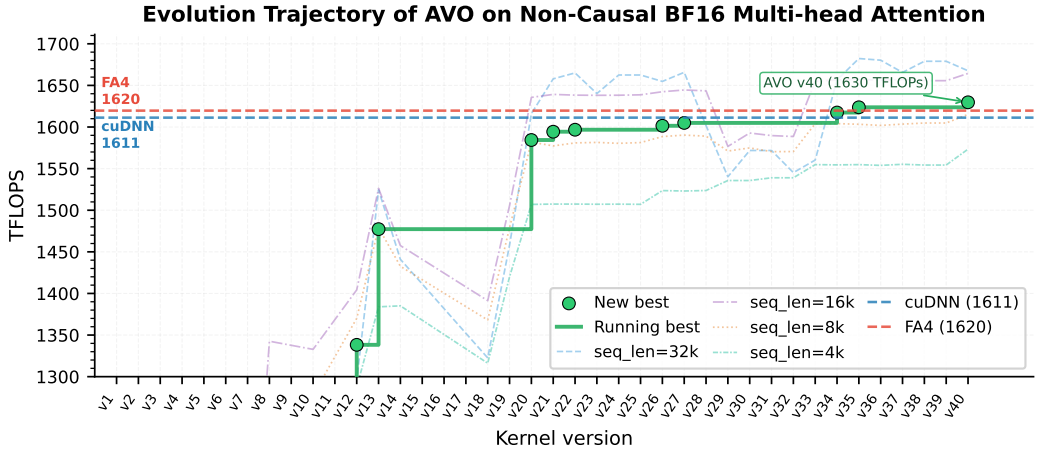


Figure 6: Evolution trajectory of AVO across 40 kernel versions over 7 days on **non-causal** MHA. The solid green line tracks the running-best geometric mean throughput across all configurations; green circles mark versions that set a new best. Dashed colored lines show per-configuration throughput (seq_len = 4k, 8k, 16k, 32k). Horizontal dashed lines indicate the geometric mean throughput of cuDNN and FA4.

that early kernel development captures coarse-grained gains while late-stage optimization squeezes out remaining headroom through increasingly fine-grained tuning.

## 5 Analysis of Agent-Discovered Optimizations

The 40-version AVO evolution produced multi-level optimizations that individually yield measurable throughput gains and collectively account for the improvements reported in Section 4. We examine three representative optimizations to illustrate the nature and depth of the agent's hardware reasoning. For each, we describe the bottleneck the agent identified in its own kernel, the change it made, and its measured impact (ablation between the version immediately before and after). Table 1 provides a summary.

Table 1: Summary of agent-discovered optimizations and their measured ablation gains (geomean TFLOPS improvement over the preceding version, across all benchmark configurations).

| Optimization | Versions | Non-causal | Causal |
|---|---|---|---|
| Branchless accumulator rescaling | v19 $\rightarrow$ v20 | $+8.1\%$ | $+1.6\%$ |
| Correction/MMA pipeline overlap | v29 $\rightarrow$ v30 | $+1.1\%$ | $+0.4\%$ |
| Register rebalancing across warp groups | v32 $\rightarrow$ v33 | $+2.1\%$ | $\sim 0\%$ |

### 5.1 Branchless Accumulator Rescaling

**Bottleneck.** In the online softmax algorithm, the running row-maximum may change as new key blocks are processed. When it does, the output accumulator $O$ must be rescaled to account for the updated maximum. In version 19 of the AVO kernel, this rescaling was implemented with a conditional branch: the kernel first checked whether any thread in the warp required rescaling, and skipped the operation entirely when the maximum was unchanged. While this avoids unnecessary computation, the branch introduces warp synchronization overhead on every iteration of the key-block loop (see Section 2.2), and the conditional control flow prevents the use of lighter memory fences in the correction path.

**AVO's approach.** In version 20, the agent replaced the conditional branch with a branchless speculative path. The rescale factor is always computed, and a predicated select substitutes 1.0 when rescaling is unnecessary; the cost of an unnecessary multiply-by-one is negligible compared to the synchronization overhead it replaces. By eliminating the branch, the agent also removed warp divergence in the correction path, which in turn allowed it to replace a blocking memory fence (which stalls until all pending memory writes complete) with a lighter non-blocking fence that merely enforces ordering. The non-blocking fence is safe here because the branchless path guarantees that all threads in the warp follow the same control flow, ensuring reconvergence before the next synchronization point.

**Measured impact.** The combined effect of branchless rescaling and the lighter fence yields $+8.1\%$ geomean throughput on non-causal and $+1.6\%$ on causal attention, the largest single optimization in the evolution. The asymmetry arises because the branchless path applies only to fully unmasked iterations of the key-block loop: non-causal attention processes all key blocks without masking, while causal attention retains the original branched logic for masked key blocks.

### 5.2 Correction/MMA Pipeline Overlap

**Bottleneck.** The attention pipeline processes two Q-tiles concurrently (dual Q-stage; see Section 2.2), each requiring a PV GEMM followed by output normalization by the correction warp. In version 29 of the AVO kernel, the two stages were serialized at the MMA-to-correction boundary: the correction warp had to wait for both PV GEMMs to complete before it could begin normalizing either stage's output, leaving it idle throughout the second GEMM.

**AVO's approach.** In version 30, the agent restructured the pipeline to allow the correction warp to begin normalizing the first stage's output as soon as its PV GEMM completes, overlapping this work with the second stage's PV GEMM. This converts a sequential dependency into a pipelined execution, reducing the idle time on the correction warp.

**Measured impact.** This pipeline restructuring yields $+1.1\%$ geomean throughput on non-causal and $+0.4\%$ on causal attention.

### 5.3 Register Rebalancing Across Warp Groups

**Bottleneck.** Blackwell partitions a fixed budget of 2048 warp-registers per SM across warp groups. In version 32 of the AVO kernel, the allocation followed the pattern of FlashAttention-4 [10]: 192 registers for the 8 softmax warps, 80 for the 4 correction warps, and 48 for the remaining 4 warps. Profiling revealed that the correction warp group was spilling values to slower local memory due to its limited 80-register budget, while the softmax group had substantial headroom.

**AVO's approach.** In version 33, the agent redistributed 8 registers from the softmax group to each of the other two groups, arriving at a 184/88/56 allocation. This redistribution is viable because the AVO kernel's softmax implementation processes score values in small fragments with packed arithmetic, resulting in a low peak register usage that leaves ample headroom even at 184 registers. The correction warp group benefits from the additional registers because, following the pipeline overlap optimization (Section 5.2), it runs concurrently with the second PV GEMM and is on the execution critical path. With 88 rather than 80 registers, fewer output values spill to local memory, reducing stalls.

**Measured impact.** Register rebalancing yields $+2.1\%$ geomean throughput on non-causal and approximately $0\%$ on causal attention.

### 5.4 Discussion

What is notable about these optimizations is that each requires jointly reasoning about multiple hardware subsystems, including synchronization and memory ordering, pipeline scheduling, and register allocation, rather than tuning any single parameter in isolation. This depth of reasoning, carried out autonomously through iterative interaction with documentation and profiling feedback, suggests that agentic variation operators can serve as an effective mechanism for expert-level kernel optimization.

## 6 Conclusion

We introduced Agentic Variation Operators (AVO), a new family of evolutionary variation operators that elevate the agent from candidate generator to variation operator. Applied to forward-pass attention on NVIDIA Blackwell GPUs, AVO produces kernels surpassing cuDNN by up to 3.5% and FlashAttention-4 by up to 10.5% over 7 days of continuous autonomous evolution. Furthermore, we show that the discovered optimizations transfer readily to grouped-query attention, requiring only 30 minutes of additional autonomous adaptation. Together, these results demonstrate that AVO can discover performance-critical micro-architectural optimizations that produce kernels surpassing state-of-the-art expert-engineered implementations. Because AVO operates at the level of variation operators rather than being tied to a specific domain, it points toward a broader path for autonomous optimization beyond attention kernels, including other performance-critical software systems on diverse hardware platforms, and engineering or scientific domains that demand extended autonomous exploration.

## Acknowledgement

We thank the NVIDIA Cutlass, cuDNN, TensorRT-LLM, FlashInfer, DevTech, IPP, and Compiler teams for valuable feedback and support. We also thank the FlashAttention-4 authors for open-sourcing their implementation and benchmark scripts, which served as a baseline and a reference for this work.

## References

[1] Michael O'Neill, Leonardo Vanneschi, Steven Gustafson, and Wolfgang Banzhaf. Open issues in genetic programming. *Genetic Programming and Evolvable Machines*, 11(3):339–363, 2010.

[2] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through large models. 2022. URL `https://arxiv.org/abs/2206.08896`.

[3] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

[4] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian,

M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

[5] Angelica Chen, David M. Dohan, and David R. So. Evoprompting: Language models for code-level neural architecture search. 2023. URL https://arxiv.org/abs/2302.14838.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2023. URL https://arxiv.org/abs/1706.03762.

[7] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. 2022. URL https://arxiv.org/abs/2205.14135.

[8] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. 2023. URL https://arxiv.org/abs/2307.08691.

[9] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. 2024. URL https://arxiv.org/abs/2407.08608.

[10] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. 2026. URL https://arxiv.org/abs/2603.05451.

[11] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cudnn: Efficient primitives for deep learning. 2014. URL https://arxiv.org/abs/1410.0759.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? 2024. URL https://arxiv.org/abs/2310.06770.

[13] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. 2024. URL https://arxiv.org/abs/2405.15793.

[14] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents. 2025. URL https://arxiv.org/abs/2407.16741.

[15] Anthropic. Claude 3.7 sonnet and claude code. https://www.anthropic.com/news/claude-3-7-sonnet, February 2025. Accessed: 2026-03-25.

[16] OpenAI. https://openai.com/index/introducing-codex/. https://openai.com/index/introducing-codex/, May 2025. Accessed: 2026-03-25.

[17] Bing Xu, Terry Chen, Fengzhe Zhou, Tianqi Chen, Yangqing Jia, Vinod Grover, Haicheng Wu, Wei Liu, Craig Wittenbrink, Wen mei Hwu, Roger Bringmann, Ming-Yu Liu, Luis Ceze, Michael Lightstone, and Humphrey Shi. VibeTensor: System Software for Deep Learning, Fully Generated by AI Agents. 2026.

[18] Chunhui Wan, Xunan Dai, Zhuo Wang, Minglei Li, Yanpeng Wang, Yinan Mao, Yu Lan, and Zhiwen Xiao. Loongflow: Directed evolutionary search via a cognitive plan-execute-summarize paradigm. 2025. URL https://arxiv.org/abs/2512.24077.

[19] Thomas Bäck, David B Fogel, and Zbigniew Michalewicz. Handbook of evolutionary computation. *Release*, 97(1):B1, 1997.

[20] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. Reevo: Large language models as hyper-heuristics with reflective evolution. 2024. URL https://arxiv.org/abs/2402.01145.

[21] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. 2015.

[22] Mert Yuksekgonul, Daniel Koceja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, and Yu Sun. Learning to discover at test time. 2026. URL https://arxiv.org/abs/2601.16175.

[23] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[24] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. 2025. URL https://arxiv.org/abs/2505.09388.

## A Comparison Using FA4-Reported Baseline Performance

Section 4 reports cuDNN and FA4 throughput measured on our hardware. In practice, minor system-level differences (driver versions, thermal conditions, clock frequencies) can affect absolute TFLOPS. Therefore, we additionally compare AVO against the cuDNN and FA4 numbers published in the FA4 paper [10]. Figure 7 presents this comparison.

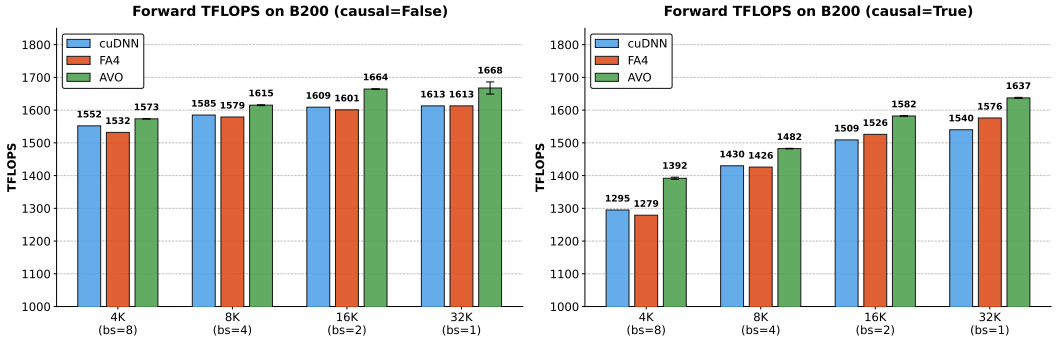


Figure 7: Multi-head attention forward-pass throughput (TFLOPS) on NVIDIA B200, comparing AVO (measured on our hardware) against cuDNN and FA4 baseline numbers as reported in the FA4 paper [10]. Head dimension 128, 16 heads, BF16. Left: non-causal. Right: causal.

On non-causal attention, AVO outperforms the FA4-reported baselines across all configurations, with gains of $+1.4\%$ to $+3.4\%$ over cuDNN and $+2.3\%$ to $+3.9\%$ over FA4. On causal attention, AVO achieves $+3.6\%$ to $+7.5\%$ over cuDNN and $+3.7\%$ to $+8.8\%$ over FA4, with the largest gains observed at shorter sequences (bs=8, seq=4096). These results are broadly consistent with the comparisons in Section 4.